

Indian Institute of Technology Kharagpur

AUTUMN Semester, 2026–2027

COMPUTER SCIENCE AND ENGINEERING

Computer Organization and Architecture Laboratory

MIPS32 Assignment 5

August 2026

Instructions: This is a group assignment. Each group must demonstrate the working programs to a Teaching Assistant for evaluation. The assignment must be submitted as a group submission through Moodle. Submit a single zipped folder named *MIPSAssignment5_{Group_Number}.zip*. The folder must contain the complete and well-commented MIPS32 assembly source files for both questions. Each source file must include a header containing the group number, the names and roll numbers of all group members, the assignment number, question number, and date of submission.

For both questions, the program must contain a `main` section and a separate recursive procedure. The `main` section must perform input, validation, procedure invocation, result printing, and program termination. The required computation must be performed entirely inside the specified recursive procedure.

Each recursive procedure must use the runtime stack correctly. In particular, it must preserve `$ra`, preserve every argument or intermediate value needed after a recursive call, restore the stack pointer before returning, and follow the standard MIPS32 register-usage conventions.

Question 1: Recursive Exponentiation by Squaring

1. Objective

Write a MIPS32 assembly-language program that computes X^N , where X is an integer base and N is a non-negative integer exponent.

The exponentiation must be implemented in a recursive procedure named

`recursive_power`.

The procedure must use exponentiation by squaring rather than performing N consecutive multiplications.

2. Problem Definition

For a non-negative exponent N , define:

$$\text{power}(X, N) = \begin{cases} 1, & N = 0, \\ \text{power}(X, \lfloor N/2 \rfloor)^2, & N > 0 \text{ and } N \text{ is even,} \\ X \cdot \text{power}(X, \lfloor N/2 \rfloor)^2, & N > 0 \text{ and } N \text{ is odd.} \end{cases}$$

For this assignment, $X^0 = 1$ for every permitted base, including 0^0 .

To keep every required test case within the signed 32-bit integer range, the accepted input range is

$$-10 \leq X \leq 10 \quad \text{and} \quad 0 \leq N \leq 9.$$

3. Required Procedure Interface

The procedure must use the following interface:

Item	Requirement
Procedure name	<code>recursive_power</code>
Input	<code>\$a0</code> contains the integer base X <code>\$a1</code> contains the non-negative exponent N
Output	<code>\$v0</code> contains X^N
Invocation	The initial call and every recursive call must use <code>jal recursive_power</code>
Return	The procedure must return using <code>jr \$ra</code>

The procedure is recursive and therefore is not a leaf procedure.

4. Recursive Algorithm

Algorithm 1 `recursive_power` using exponentiation by squaring

Require: Integer base X and non-negative exponent N

Ensure: Return X^N

```

1: if  $N = 0$  then
2:   return 1
3:  $H \leftarrow \text{RECURSIVE\_POWER}(X, \lfloor N/2 \rfloor)$ 
4:  $R \leftarrow H \times H$ 
5: if  $N$  is odd then
6:    $R \leftarrow R \times X$ 
7: return  $R$ 

```

Only one recursive call may be made in each non-base-case invocation. The parity of N should be tested using a bitwise operation, and $\lfloor N/2 \rfloor$ should be obtained using a logical right shift.

5. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter the integer base X .
2. Read the base using syscall 5.
3. Ask the user to enter the non-negative exponent N .
4. Read the exponent using syscall 5.
5. Validate in `main` that $-10 \leq X \leq 10$ and $0 \leq N \leq 9$.
6. Print an error message and terminate if either input is invalid.
7. Preserve the original input values in `main` for output.

8. Place X in `$a0` and N in `$a1`.
9. Call the procedure using `jal recursive_power`.
10. Perform the complete exponentiation inside `recursive_power`.
11. Return the result through `$v0`.
12. Print the result from `main`.
13. Terminate the program using `syscall 10`.

6. Required Stack Behaviour

For every non-base-case invocation, `recursive_power` must:

1. allocate a stack frame before making the recursive call;
2. save `$ra`;
3. save the original base and exponent, or otherwise preserve all values needed after the recursive call;
4. call itself using `jal recursive_power`;
5. use the returned value to compute the final result;
6. restore all saved values;
7. deallocate the complete stack frame; and
8. return using `jr $ra`.

A value held only in a `$t` register must not be assumed to survive a recursive call.

7. Suggested Control-Flow Organization

```
.data
    # Base prompt
    # Exponent prompt
    # Result message
    # Invalid-input message
    # Newline string

.text
.globl main

main:
    # Read X
    # Read N
    # Validate both inputs
    # Preserve the original values
    # Place X in $a0 and N in $a1

    jal recursive_power

    # Obtain the result from $v0
    # Print X, N, and the result
    # Terminate using syscall 10
```

```

recursive_power:
    # Input:  $a0 = X, $a1 = N
    # Output: $v0 = X^N

    # Test the base case
    # Create a stack frame
    # Save $ra and values needed after recursion
    # Recursively compute power(X, floor(N/2))
    # Square the returned value
    # Multiply once more by X when N is odd
    # Restore the stack frame

    jr $ra

invalid_input:
    # Print the error message
    # Terminate using syscall 10

```

The listing specifies only the required organization. Students must complete the MIPS32 instructions independently.

8. Input Validation

The following inputs are valid:

$$-10 \leq X \leq 10, \quad 0 \leq N \leq 9.$$

Any base outside the permitted range or any exponent outside the permitted range is invalid. For invalid input, the program must print a message similar to:

Error: base must be in [-10, 10] and exponent must be in [0, 9].

Input validation must be completed in `main` before calling `recursive_power`.

9. Allowed Instructions and Directives

The solution may primarily use:

- `.data`, `.text`, `.globl`, and `.ascii`;
- `la`, `li`, and `move`;
- `add`, `addi`, `addu`, and `addiu`;
- `sub` and `subu`;
- `andi` and `srl`;
- `mul`, or equivalently `mult` followed by `mflo`;
- `lw` and `sw`;
- `beq`, `bne`, `bgt`, `bge`, `blt`, `ble`, and `bltz`;
- `b`;
- `jal`;
- `jr`; and

- `syscall`.

Equivalent pseudo-instructions supported by MARS or QtSPIM may be used.

11. Sample Runs

Sample Run 1

```
Enter the base: 2
Enter the exponent: 8
2 raised to the power 8 is: 256
```

Sample Run 2

```
Enter the base: -3
Enter the exponent: 5
-3 raised to the power 5 is: -243
```

Sample Run 3

```
Enter the base: 7
Enter the exponent: 0
7 raised to the power 0 is: 1
```

Sample Run 4: Invalid Input

```
Enter the base: 4
Enter the exponent: -2
Error: base must be in [-10, 10] and exponent must be in [0, 9].
```

12. Required Test Cases

Base X	Exponent N	Expected value in $\$v0$
2	0	1
0	0	1
0	5	0
2	5	32
3	4	81
-2	5	-32
-2	6	64
5	7	78125
10	9	100000000
11	2	Error message
2	10	Error message
2	-1	Error message

Additional test cases may be used during evaluation.

13. Correctness Requirements

A correct implementation must satisfy the following conditions:

1. Both inputs are read and validated in `main`.
2. Invalid inputs are rejected before the recursive procedure call.
3. The original inputs are preserved for result printing.
4. The base is passed through `$a0` and the exponent through `$a1`.
5. The procedure is called using `jal recursive_power`.
6. The base case $N = 0$ returns 1.
7. Every non-base-case invocation makes exactly one recursive call using exponent $\lfloor N/2 \rfloor$.
8. Parity is tested using a bitwise operation.
9. The procedure preserves `$ra` and all required values on the stack.
10. The stack pointer has the same value on entry to and return from each invocation.
11. The result is returned through `$v0`.
12. The final output is printed by `main`.
13. The program terminates using syscall 10.

14. Submission Requirement for Question 1

The source file for Question 1 must be named:

`recursive_power_{Group_Number}.S`

The file must begin with a comment header similar to:

```
# -----  
# Group Number:  
# Group Members (Name and Roll Number):  
# 1.  
# 2.  
# 3.  
# Course: Computer Organization and Architecture Laboratory  
# Assignment: MIPS32 Assignment 5  
# Question: 1  
# Problem: Recursive exponentiation by squaring  
# Date:  
# -----
```

Question 2: Find the Maximum Element of an Array Recursively

1. Objective

Write a MIPS32 assembly-language program that finds the maximum element of a signed integer array.

The maximum-element computation must be implemented in a recursive procedure named

`find_max`.

2. Problem Definition

Let A be an array containing N signed integers. The maximum may be defined recursively as

$$\text{find_max}(A, N) = \begin{cases} A[0], & N = 1, \\ \max(\text{find_max}(A, N - 1), A[N - 1]), & N > 1. \end{cases}$$

The program must accept an array size in the range

$$1 \leq N \leq 20.$$

The array elements may be positive, zero, or negative signed integers.

3. Required Procedure Interface

The procedure must use the following interface:

Item	Requirement
Procedure name	<code>find_max</code>
Input	<code>\$a0</code> contains the base address of the array <code>\$a1</code> contains the number of elements N
Output	<code>\$v0</code> contains the maximum signed integer in the array
Invocation	The initial call and every recursive call must use <code>jal find_max</code>
Return	The procedure must return using <code>jr \$ra</code>

The procedure is recursive and therefore is not a leaf procedure.

4. Recursive Algorithm

Algorithm 2 `find_max` procedure

Require: Base address of array A and number of elements $N \geq 1$

Ensure: Return the maximum element among $A[0], \dots, A[N - 1]$

```
1: if  $N = 1$  then  
2:   return  $A[0]$   
3:  $M \leftarrow \text{FIND\_MAX}(A, N - 1)$   
4:  $V \leftarrow A[N - 1]$   
5: if  $V > M$  then  
6:   return  $V$   
7: else  
8:   return  $M$ 
```

5. Assignment Task

Write a MIPS32 assembly-language program that performs the following operations:

1. Ask the user to enter the array size N .
2. Read N using syscall 5.
3. Validate in `main` that $1 \leq N \leq 20$.
4. Print an error message and terminate if N is invalid.
5. Read exactly N signed integers into an array in memory.
6. Place the array base address in `$a0`.
7. Place N in `$a1`.
8. Call the procedure using `jal find_max`.
9. Perform the complete maximum-element computation inside `find_max`.
10. Return the maximum value through `$v0`.
11. Print the returned maximum value from `main`.
12. Terminate the program using syscall 10.

6. Required Stack Behaviour

For every non-base-case invocation, `find_max` must:

1. allocate a stack frame before making the recursive call;
2. save `$ra`;
3. preserve the array base address and the original value of N , or otherwise preserve all values needed after recursion;
4. call itself using `jal find_max` with $N - 1$;
5. obtain $A[N - 1]$ after the recursive call;
6. compare $A[N - 1]$ with the recursively returned maximum using signed comparison;
7. restore all saved values;
8. deallocate the complete stack frame; and
9. return using `jr $ra`.

7. Suggested Control-Flow Organization

```
.data
    array: .space 80          # Space for 20 words
    # Size prompt
    # Element prompt
    # Result message
    # Invalid-size message
    # Newline string

.text
.globl main
```



```

main:
    # Read N
    # Validate 1 <= N <= 20
    # Read N signed integers into array
    # Place array address in $a0 and N in $a1

    jal find_max

    # Obtain the maximum from $v0
    # Print the result
    # Terminate using syscall 10

find_max:
    # Input:  $a0 = array base address
    #         $a1 = number of elements
    # Output: $v0 = maximum element

    # Test the base case N = 1
    # Create a stack frame
    # Save $ra and values needed after recursion
    # Recursively compute find_max(array, N - 1)
    # Load array[N - 1]
    # Perform a signed comparison
    # Restore the stack frame

    jr $ra

invalid_size:
    # Print the error message
    # Terminate using syscall 10

```

The listing specifies only the required organization. Students must complete the MIPS32 instructions independently.

8. Input Validation

The array size must satisfy

$$1 \leq N \leq 20.$$

The following sizes are invalid:

- zero;
- any negative integer; and
- any integer greater than 20.

For an invalid size, the program must print a message similar to:

Error: array size must be between 1 and 20.

Individual array elements do not require range validation beyond being valid signed integers accepted by syscall 5.

9. Allowed Instructions and Directives

The solution may primarily use:

- .data, .text, .globl, .ascii, and .space;
- la, li, and move;
- add, addi, addu, and addiu;
- sub and subu;
- sll;
- lw and sw;
- slt, beq, bne, bgt, bge, blt, and ble;
- b;
- jal;
- jr; and
- syscall.

Equivalent pseudo-instructions supported by MARS or QtSPIM may be used.

11. Sample Runs

Sample Run 1

```
Enter the number of elements: 5
Enter element 1: 7
Enter element 2: 3
Enter element 3: 12
Enter element 4: 4
Enter element 5: 9
The maximum element is: 12
```

Sample Run 2

```
Enter the number of elements: 4
Enter element 1: -8
Enter element 2: -3
Enter element 3: -15
Enter element 4: -6
The maximum element is: -3
```

Sample Run 3

```
Enter the number of elements: 1
Enter element 1: 42
The maximum element is: 42
```

Sample Run 4: Invalid Size

```
Enter the number of elements: 0
Error: array size must be between 1 and 20.
```

12. Required Test Cases

Size	Array elements	Expected value in \$v0
1	7	7
4	3, 1, 9, 2	9
3	-5, -2, -9	-2
4	-4, 0, -1, -7	0
5	8, 8, 8, 8, 8	8
6	12, -3, 25, 4, 25, 10	25
5	-1, -20, -3, -4, -5	-1
0	No elements	Error message
21	Not required	Error message
-3	No elements	Error message

Additional test cases may be used during evaluation.

13. Correctness Requirements

A correct implementation must satisfy the following conditions:

1. The array size is read and validated in `main`.
2. Invalid sizes are rejected before reading array elements.
3. Exactly N signed integers are stored in consecutive word locations.
4. The array base address is passed through `$a0` and N through `$a1`.
5. The procedure is called using `jal find_max`.
6. The base case $N = 1$ returns $A[0]$.
7. Every non-base-case invocation recursively processes the first $N - 1$ elements.
8. The address of $A[N - 1]$ is computed correctly using a four-byte word offset.
9. The comparison is signed and works correctly for arrays containing only negative values.
10. The procedure preserves `$ra` and all required values on the stack.
11. The stack pointer has the same value on entry to and return from each invocation.
12. The maximum is returned through `$v0`.
13. The final result is printed by `main`.
14. The program terminates using `syscall 10`.

14. Submission Requirement for Question 2

The source file for Question 2 must be named:

`recursive_max_{Group_Number}.S`

The file must begin with a comment header similar to:

```
# -----
# Group Number:
# Group Members (Name and Roll Number):
# 1.
# 2.
# 3.
# Course: Computer Organization and Architecture Laboratory
# Assignment: MIPS32 Assignment 5
# Question: 2
# Problem: Recursive maximum element of an array
# Date:
# -----
```

Common Submission Requirements

The assignment must be submitted as a group submission through Moodle. Only one submission is required from each group.

Submit a single zipped folder named:

MIPSAssignment5_{Group_Number}.zip

The folder must contain the following files:

File	Contents
recursive_power_{Group_Number}.S	Solution for Question 1: Recursive Exponentiation by Squaring
recursive_max_{Group_Number}.S	Solution for Question 2: Find the Maximum Element of an Array Recursively